

끝까지 완성하는 개발자

박성희

SEONGHEE PARK

Unity Client Programmer

학교 · 전공	유한대학교 게임콘텐츠전공 · 2027 졸업 예정
주 스택	Unity · C#, Photon, Git, Unity Profiler, Claude Code
자격증	워드프로세서 2022 · 대한상공회의소 정보처리기능사 2023 · 한국산업인력공단

GitHub ↗ | 이메일 | PDF 다운로드



실제 플레이 화면

PROJECT 01 · 액션 로그라이크

로그타워 ROGUE TOWER

절차적으로 생성되는 던전에서 스킬과 장비를 조합해 보스까지 살아남는 로그라이크 액션

장르	액션 로그라이크 (액션 · 전략)
엔진·언어	Unity 6 / C#
개발 기간	약 3.5개월
팀 구성	3인 (본인 포함)
출시	STOVE · 2026.04.28

프로젝트 자세히 보기 →

STOVE 스토어 ↗ GitHub 코드 ↗

담당

스킬 · 맵 · 상자 · 상점 · 인벤토리

개발 90%

디자인 10%

핵심 수치

Unity Profiler 실측 · 최대 스택 나선 탄막 패턴 기준

166개

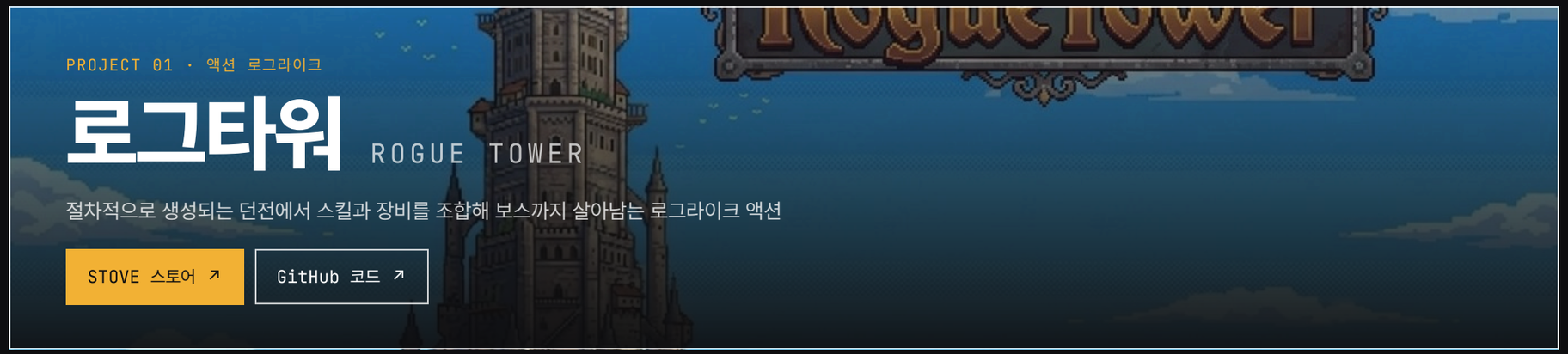
동시 활성 탄막 — 실측 피크

2.9ms → 4.7ms

GPU 프레임 타임 · 평상시 → 최대 부하 (+62%)

40 → 228

오브젝트 풀 크기 · 사전 생성 후 동적 확장 (Instantiate 188회)



PROJECT 01 · 액션 로그라이크

로그타워 ROGUE TOWER

절차적으로 생성되는 던전에서 스킬과 장비를 조합해 보스까지 살아남는 로그라이크 액션

STOVE 스토어 ↗ | GitHub 코드 ↗

CUT 01 스킬 시스템

ScriptableObject로 스킬을 데이터로 분리해, **코드 수정 없이 스킬 추가와 밸런스 조정**이 가능합니다.

예시 · 스나이퍼 스킬

- **Lv.1** Q 키 → 적을 관통하는 총알 발사
- **Lv.2** 성장 시 강력한 한 발 (스나이퍼 모드)



스킬 발동 화면

SkillData.cs

```
스킬을 코드가 아닌 데이터(ScriptableObject)로 관리
[CreateAssetMenu(fileName = "NewSkill", menuName = "Skill/Skill Data")]
public class SkillData : ScriptableObject
{
    [Header("식별자 (PlayerPrefs 키로 사용· 유니크해야 함)")]
    public string skillId;

    [Header("표시 정보")]
    public string skillName;
    public Sprite icon;
    [TextArea(2, 4)] public string description;

    [Header("비용 (레벨 잼)")]
    public int cost = 20;

    [Header("진화 레벨 (Lv1, Lv2, Lv3... 순서대로 채워넣기)")]
    public SkillLevel[] levels;
}
```

SkillData.cs — ScriptableObject 스킬 정의

SkillManager.cs

```
공격력*공속이 조건을 넘으면 레벨 해금 (TryUnlockLevels 발췌)
            if (atk >= lv.requiredAtk && spd >= lv.requiredAtkSpeed)
            {
                unlockedLevels[s.skillId] = lv.level;
                current = lv.level;
                anyUnlocked = true;
```

SkillManager.cs — 조건 충족 시 레벨 해금

CUT 02 맵 시스템

층마다 미리 만든 맵 중 하나를 무작위로 불러오고, 몬스터를 모두 처치하면 다음 층 문이 열립니다.

> 구조를 바꾼 이유
>
> 처음에는 20개 층을 하나씩 손으로 만들 계획이었지만, 일정과 아이디어가 부족해진다는 기획자의 제안을 받아 미리 만든 맵을 무작위로 불러오는 방식으로 바꿨습니다.

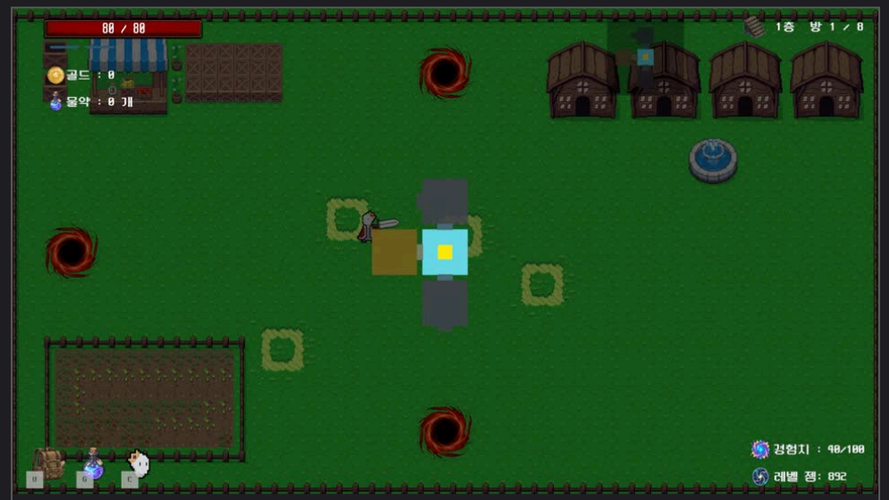
한 층이 진행되는 순서

01 맵 생성
1~9층은 맵 목록에서 무작위 선택, 10층은 보스 맵

02 길 찾기 준비
맵이 만들어진 뒤 몬스터 이동 경로(NavMesh)를 계산

03 몬스터 배치
스폰 위치마다 몬스터 종류를 무작위로 골라 배치

04 클리어 판정
몬스터를 모두 처치하면 보상과 함께 다음 층 문이 열림



층 이동 화면

MapManager.cs — GoToNextMap() · STOVE 출시 버전

```
77 if (currentFloor == maxFloorBeforeBoss + 1) // 보스방
...
81     mapToSpawn = currentStageData.bossMapPrefab;
...
86 else // 일반방
...
90     int idx = Random.Range(0, currentStageData.normalMapPrefabs.Length);
91     mapToSpawn = currentStageData.normalMapPrefabs[idx];
```

MapManager.cs — 10층은 보스 맵, 1~9층은 일반 맵 중 무작위 선택

CUT 03 데이터 설계 & 최적화

함께 일하는 기획자와, 무거운 전투 구간을 위해 만든 구조

JSON 데이터 분리

몬스터 능력치를 코드에 하드코딩하지 않고 JSON으로 분리했습니다. 기획자가 코드 수정 없이 밸런스를 조절할 수 있는 구조입니다.

MonsterStat.json

JSON 원본 데이터 (발췌)

```
[
  {
    "MonsterID": "boss1",
    "MonsterName": "미노타우로스",
    "CombatType": "Boss",
    "Element": "Normal",
    "HP": 1800,
    "ATK": 20,
    "DEF": 0,
```

DataManager.cs — JSON을 Dictionary로 변환 후 조회 (발췌)

```
54      public MonsterData GetMonsterData(string id)
55      {
56          if (monsterDict.TryGetValue(id, out MonsterData data))
57          {
58              return data;
```

MonsterStat.json · DataManager

오브젝트 풀링

보스 탄막처럼 부하가 큰 구간에서 생성·파괴 비용을 줄여 프레임 저하를 막았습니다.

PoolManager.cs

Get() — 새로 만들지 않고 꺼내서 재사용

```
79      if (poolDictionary[type].Count > 0)
80      {
81          GameObject obj = poolDictionary[type].Dequeue();
82          obj.transform.position = position;
83          obj.transform.rotation = rotation;
84          obj.SetActive(true);
85          return obj;
86      }
```

Return() — 파괴하지 않고 큐에 반납

```
105   public void Return(PoolItemType type, GameObject obj)
106   {
107       obj.SetActive(false);
108       poolDictionary[type].Enqueue(obj);
109   }
```

PoolManager.cs

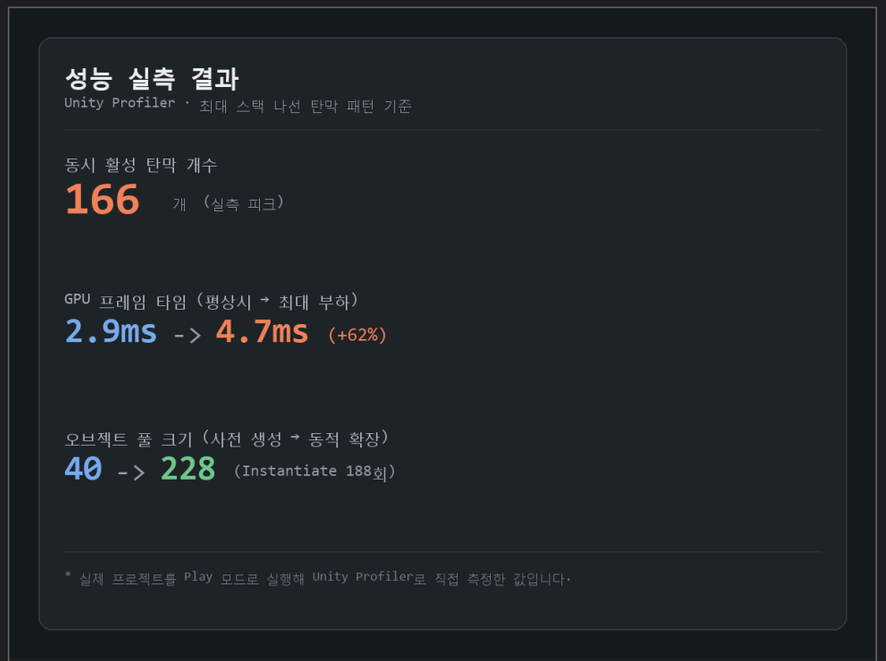
CUT 04 **성능 실측 결과**

Unity Profiler로 직접 측정 · 최대 스택 나선 탄막 패턴 기준

166개
동시 활성 탄막 — 실측 피크, 대량 탄막에도 안정 유지

2.9ms → **4.7**ms
GPU 프레임 타임 · 평상시 → 최대 부하 (+62%)

40 → **228**
오브젝트 풀 크기 · 사전 생성 후 동적 확장 (Instantiate 188회)



Unity Profiler 측정 결과

CUT 05 트러블슈팅 — 인벤토리 능력치 연결 버그

문제 장비를 장착해도 공격력·이동속도가 오르지 않음. 다른 팀원이 만들던 인벤토리를 넘겨받은 상태라 원인이 한눈에 보이지 않음.

원인 장비 능력치를 캐릭터가 읽지 않는 `attackScript`(인벤토리 공격력)에 저장하고 있었음.

해결 캐릭터가 실제로 읽는 `playerScript` 변수에 '인벤토리 공격력 + 플레이어 공격력' 최종값을 전달하도록 수정.

> **핵심** 버그는 계산이 아니라 '어디에 저장하느냐'의 문제. 남의 코드도 끝까지 추적해 원인을 찾았습니다.

InventoryManager.cs — BEFORE · 버그

```
장비 스탯을 attackScript(사용되지 않는 컴포넌트)에 저장 → 캐릭터 능력치 미반영
296         else if (playerWeaponTransform != null) playerWeaponTransform.localScale = new Vector3(2f, 2f, 2f);
297
298         if (attackScript != null)
299         {
300             attackScript.WeaponBonusDamage = totalAtk;
301             attackScript.WeaponbonusAttackSpeed = totalAtkSpd;
302         }
303
304         if (playerScript != null)
```

수정 전

InventoryManager.cs — AFTER · 수정

```
캐릭터의 TotalAtk 계산식이 실제로 읽는 playerScript 변수에 저장 → 정상 반영
321         // * [수정] attackScript -> playerScript (또는 player)
322         if (playerScript != null)
323         {
324             // * [수정] WeaponBonusDamage -> WeaponBonusAtk
325             playerScript.WeaponBonusAtk = totalAtk;
326
327             // * [수정] WeaponbonusAttackSpeed -> WeaponBonusAtkSpeed
328             playerScript.WeaponBonusAtkSpeed = totalAtkSpd;
329         }
```

수정 후

CUT 06 출시 후 유저 피드백 반영

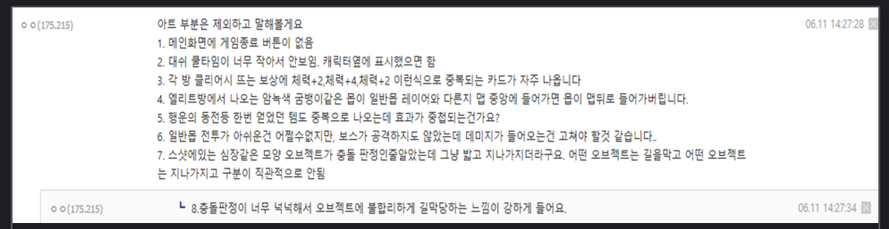
출시로 끝내지 않고, 커뮤니티에 남은 피드백을 받아 게임을 다듬었습니다.

"대쉬 쿨타임이 작아 안 보인다"

→ 쿨타임을 캐릭터 옆 UI로 시각화

"충돌 판정이 넉넉해 길막당한다"

→ 오브젝트 충돌 판정 범위 조정



실제 유저 피드백 (STOVE 커뮤니티 게시판)

> **배운 점** 만드는 사람은 놓치지만, 플레이하는 사람은 바로 느끼는 지점이 있다는 것을 배웠습니다.



딜러 애니메이션

PROJECT 02 · 심리전 카드 게임

ZooJack

딜러의 속임수를 고발해 승패를 뒤집는 3인 심리전 카드 게임

장르	카드 게임 · 전략 (블랙잭 기반)
엔진·언어	Unity / C#
네트워크	Photon 멀티플레이
팀 구성	2인 (클라이언트 프로그래머)
개발 기간	2026.06 ~ 2026.08
출시	itch.io · 2026.08.22

프로젝트 자세히 보기 →

itch.io에서 플레이 ↗ | GitHub 코드 ↗

담당

Photon 멀티플레이 · 카드 로직

개발	90%
디자인	10%

핵심 수치

3인

Photon으로 같은 게임 상태를 공유하는 심리전

1 → 4

GameDirector의 화면 표시 코드를 4개 클래스로 분리
(Presenter 1 · View 3)



PROJECT 02 · 심리전 카드 게임

ZooJack

딜러의 속임수를 고발해 승패를 뒤집는 3인 심리전 카드 게임

itch.io에서 플레이 ↗ GitHub 코드 ↗

CUT 01 게임 화면







CUT 02 고발 시스템

딜러의 속임수를 눈치채고 고발해 승패를 뒤집는 심리전 장치

01 딜러가 카드를 조작
라운드 진행 중 딜러는 몰래 카드를 바꿀 수 있다

02 패배한 플레이어가 고발
진 쪽이 딜러의 조작을 의심하면 고발을 선택

03 고발 판정
조작이 있었으면 고발 성공, 없었으면 실패

04 승패 역전 또는 유지
성공 시 결과가 뒤집히고, 실패 시 그대로 패배

내가 구현한 부분

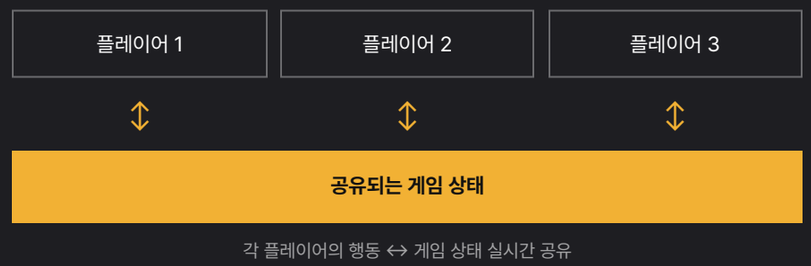
- 카드 게임의 규칙과 고발 로직 구현
- 고발 성공·실패에 따라 승패 결과를 전환하는 처리
- Photon으로 여러 플레이어가 같은 게임 상태를 공유하도록 동기화



실제 고발 화면 — 진 플레이어가 딜러를 고발

CUT 03 Photon 멀티플레이

한 사람의 행동이 다른 사람 화면에도 똑같이 반영되도록 게임 상태를 맞췄습니다.



각 플레이어의 행동 ↔ 게임 상태 실시간 공유

내가 맡은 부분

- 게임 방을 만들고 입장하는 기능
- 카드 분배·턴 진행 같은 상태를 플레이어끼리 맞추기
- 한 명의 행동이 다른 화면에도 반영되도록 동기화

CUT 04 트러블슈팅 — GameDirector 리팩터링

문제 게임 진행 로직과 화면 표시(UI) 코드가 GameDirector 한 클래스에 몰려, 인스펙터에서 값을 찾고 수정하기가 점점 어려워짐.

원인 게임 규칙 처리와 칩·머니바·승패 표시 같은 UI 갱신이 한 클래스에 섞여 책임이 과도하게 집중됨. 하드코딩된 값도 관리가 힘들어짐.

해결 화면 표시 코드를 역할별로 4개 클래스(Presenter 1개 · View 3개)로 분리하고, 하드코딩을 걷어냄.

결과 기능별 위치가 명확해져 값 수정과 디버깅이 쉬워짐. '동작하는 코드'와 '유지보수되는 코드'의 차이를 체감.

Before

GameDirector
게임 진행 + 화면 표시가 한 클래스에

↓ After — 화면 표시 코드를 역할별로 분리

GamePanelPresenter
핫시트·네트워크 공통 화면 조작

BetChipsView
판돈 칩·금액 표시

MoneyBarView
플레이어별 머니바

ResultMarkerView
승패 표식 (왕관·비석)

게임 진행은 GameDirector에 남기고, 화면 표시만 4개 클래스로 분리

ABOUT · 안녕하세요, Unity 클라이언트 프로그래머

이런 걸 잘해요

로그라이크 **로그타워**와 멀티플레이 카드 게임 **ZooJack**을 팀으로 개발해 STOVE와 itch.io에 출시했습니다. 콘텐츠가 계속 쌓이는 게임을 데이터 분리 구조와 최적화로 안정적으로 확장하는 일에 관심이 있습니다.

출시까지 완성한 두 프로젝트에서 직접 구현하고 검증한 역량입니다.

데이터 분리 설계

스킬은 ScriptableObject, 몬스터 능력치는 JSON으로 분리해 코드 수정 없이 콘텐츠 추가

협업 · 소통

회의 기록 채널과 레퍼런스 이미지로 팀의 작업 방향을 하나로 정리

디버깅

넘겨받은 남의 코드도 끝까지 추적해 버그의 원인을 찾아 해결

성능 최적화

보스 탄막 구간에 오브젝트 풀링 적용, Unity Profiler로 결과를 직접 측정

학력

2021 원미고등학교 졸업

2021 유한대학교 게임콘텐츠전공 입학

2027 유한대학교 게임콘텐츠전공 졸업 예정

자격증

2022 워드프로세서 대한상공회의소

2023 정보처리기능사 한국산업인력공단

2026 구글 AI 프로그램 수료 ↗

출시 프로젝트

2026 로그타워 · STOVE 3인

2026 ZooJack · itch.io 2인

EXPERIENCE

도전과 성장의 기록

고등학교 시절

전공으로의 첫 발

방과 후 타 학교에서 열리는 교육 프로그램에 자발적으로 참여해 한 학기 동안 아두이노와 C 코딩을 배웠습니다. 새로운 분야에 먼저 뛰어든 이 경험은, 낯선 기술도 일단 부딪혀 익히는 지금의 학습 태도로 이어졌습니다.

고등학교 · 코딩 동아리

가위바위보 텍스트 게임

친구들과 직접 코딩 동아리를 만들어 텍스트 게임을 제작했습니다. 구현 중 막히는 부분이 많았지만, 대화와 검색으로 원인을 좁혀가며 끝까지 해결했습니다. 이때 익힌 '모르면 함께 찾아 해결한다'는 태도를 지금도 팀 협업과 디버깅에 그대로 적용하고 있습니다.

대학교 · Unity 동아리

위기를 정면돌파한 마감

발표 과제 마감이 코앞인데 디자인 리소스가 부족하고 팀이 멈춘 상황이었습니다. "디자인은 내가 구해오겠다"며 먼저 해결책을 제안하고, 팀원이 막힌 부분은 직접 맡아 마감을 지켜냈습니다. 이 경험으로 위기에서 주도적으로 역할을 찾아 움직이는 법을 배웠고, 실무에서도 병목을 먼저 파악해 해결하는 자세로 이어가려 합니다.

2026 · 구글 AI 프로그램

구글 AI 프로그램 최종 수료

2026년 구글 AI 프로그램을 최종 수료하여 AI의 사용법을 더욱 자세히 알게 되었습니다.

📄 수료증 보기 ↗

2026.06.24 ~ 07.07 · 바이브 코딩 방학 특강(유한대)

개발 도구로 나를 담다

방학 특강에 참여해 GitHub 업로드, Claude Code, Obsidian 사용법을 익히고, 자기소개 웹페이지를 직접 만들었습니다. 또한 AI를 활용해 자기소개 영상을 제작하고, 로블록스 멀티플레이 환경에서 즐기는 게임도 직접 만들어 보았습니다.



■ 게임 화면

GAME · Roblox · Multiplayer

로블록스로 만든 멀티플레이 카지노 게임

개발 기간	바이브 코딩 방학 특강
팀 구성	1인 개발 (전 과정)
엔진·언어	Roblox / Luau

왜 만들었나

여러 명이 함께 즐기는 카지노 경험에 대한 수요를 바탕으로, 로블록스 멀티플레이 환경에서 동작하는 게임이 필요하다고 보고 직접 구현했습니다.

할 수 있는 것

로블록스(Luau) 멀티플레이 환경에서 동작하는 게임을 기획부터 구현까지 직접 만들 수 있습니다.

PLAY HISTORY

게임 플레이 이력

플레이 시간이 긴 순서로 정리하고, 게임마다 설계 측면에서 인상 깊었던 점을 한 줄로 남겼습니다.

37종
PC (Steam) · 누적 약 771시간

26종
모바일 · Play 게임즈 기록 153종 중 상위

8종
로그라이크 계열 · 로그타워 개발에 직접 참고

PC · Steam | 모바일

게임 이름이나 장르로 찾기

1 **Palworld** ↗ 오픈월드 서바이벌
121.4시간
팰 포획·육성과 기지 자동화를 결합한 서바이벌로, 수집 욕구와 생산 루프가 맞물려 플레이를 계속 이어가게 만드는 구조가 인상적임

2 **Skul: The Hero Slayer** ↗ 액션 로그라이크
86.7시간
머리를 교체하면 스킬 세트 전체가 바뀌는 구조로, 하나의 조작 체계 위에서 100여 가지 플레이 스타일을 만들어낸 설계가 인상적임

3 **철권 8** ↗ 대전 격투
83.7시간
프레임 단위 수 싸움이 핵심인 3D 대전 격투로, 캐릭터별 패턴을 익힐수록 실력이 눈에 보이게 쌓이는 재미가 있음

4 **Baldur's Gate 3** ↗ CRPG
69.3시간
선택에 따라 전개가 달라지는 RPG로, 규칙 기반 시스템이 플레이어의 예상 밖 발상까지 대부분 수용해주는 설계가 인상적임

5 **마블 스냅** ↗ 디지털 CCG
54.7시간
12장 덱·6턴이라는 제약으로 한 판을 3분에 끝내는 카드 게임으로, 짧은 호흡 안에 심리전을 압축해 넣은 설계가 인상적임

6 **Slay the Spire 2** ↗ 덱빌딩 로그라이크
46.1시간
카드 조합으로 매 판 다른 빌드를 완성하는 게임으로, 랜덤하게 주어진 선택지 안에서 시너지를 설계하는 재미가 있음

데이터 출처 +

◀ 1 / 7 ▶

GOALS

앞으로의 계획

1인으로 개발 중인 3D 프로젝트를 플레이 가능한 빌드로 완성해, 전투 시스템 설계와 성능 최적화 경험을 쌓겠습니다

취업 목표

2027년 상반기 · 라이브 서비스 게임 개발사 · Unity 클라이언트 프로그래머 (신입)

지금 가진 강점

- 게임 2개 출시 (STOVE · itch.io)
- 데이터 분리 구조 설계 (ScriptableObject · JSON)
- Profiler로 실측한 최적화

채워야 할 점

- UI · 연출 구현 경험

1단계 3D 개인 프로젝트 완성 진행 중

졸업 전 2026.10 ~ 2027.02



개발 중인 실제 전투 화면

개요

Unity 6 / C# · 1인 개발 · 부대 지휘 생존 게임

병력 편성 · 병사 성장 · 카드 선택으로 웨이브를 버티는 구조

현재 구현

- 병사별 전투와 스킬, 지휘관 지원 카드
- Day별 웨이브와 보스 돌진 패턴

완성 계획과 결과물

- 전투 밸런스·UI 정리 → **처음부터 끝까지 플레이 가능한 빌드**
- Profiler로 병목 측정 → **개선 전후 수치 기록**

2단계 입사 1년 차

맡은 게임의 코드 구조와 데이터 흐름을 익혀, 업데이트 콘텐츠를 버그 없이 구현

3단계 입사 2~3년 차

장비·스킬·스테이지를 데이터만으로 추가하는 구조를 설계하고 배포까지 담당

CONTACT

함께 일하고 싶으시다면 편하게 연락 주세요

E-mail	aprk5678@naver.com
GitHub	github.com/parkseonghee
STOVE	로그타워
itch.io	ZooJack

이메일 보내기 | GitHub ↗ | PDF 다운로드

END

감사합니다! 잘 부탁드립니다 🙂

처음으로 돌아가기 ↺